

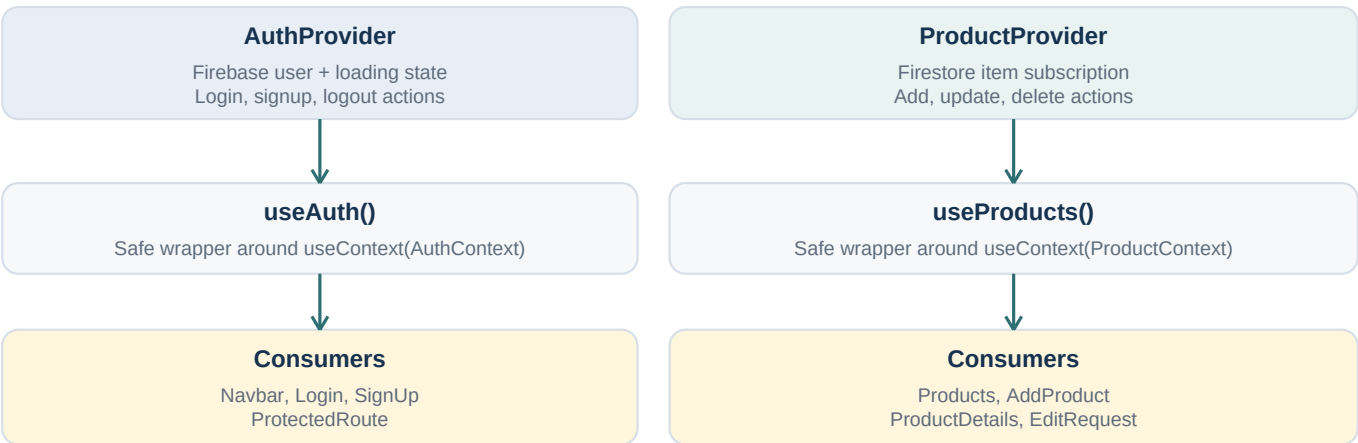
React Hooks Explained

Where and why the project uses `useState`, `useEffect`, `useContext`, `useMemo`, `useCallback`, `useRef`, custom hooks, and React Router hooks.

Scope: ItemVault source code reviewed on 15 June 2026. Examples and file references are taken from the project, not from a generic demo.

The big idea

Hooks let function components remember state, react to external changes, share data, and interact with routing. ItemVault uses them in two layers: providers manage shared Firebase data, while pages and components manage local UI state.



Quick usage map

React feature	Where ItemVault uses it	Main job
<code>useState</code>	Auth and product providers; forms; filters; theme; delete state	Stores values that change and should update the UI
<code>useEffect</code>	Firebase Auth; Firestore; theme persistence; edit-page title	Synchronizes React with external systems
<code>useContext</code>	<code>useAuth</code> and <code>useProducts</code>	Reads shared provider data without prop drilling
<code>useMemo</code>	Provider values and inventory filtering/options	Reuses calculated values until dependencies change
<code>useCallback</code>	Product add, update, delete, and lookup functions	Keeps function identities stable for context consumers
<code>useRef</code>	Pending deletions and duplicate form submission guard	Stores mutable values without causing a render
Router hooks	<code>useNavigate</code> , <code>useParams</code> , <code>useLocation</code>	Navigation, URL parameters, and route state

1. useState - values that change on screen

What it means: useState gives a component a value and a setter. Calling the setter schedules a new render so the interface reflects the latest value.

Why ItemVault uses it: It manages authentication status, Firestore products, loading flags, form fields, search/filter controls, theme selection, and submission or deletion feedback.

```
const [products, setProducts] = useState([]);
const [loading, setLoading] = useState(true);

const [search, setSearch] = useState("");
const [statusFilter, setStatusFilter] = useState("all");
```

Source: src/context/ProductContext.jsx and src/pages/Products.jsx, lines 19-20 and 26-30

Important: State is used only when a value must survive renders and affect rendered output. Derived values such as filteredItems are calculated from state instead of being stored as extra state.

Main useState groups in ItemVault

Area	Examples	Why state is appropriate
Authentication	user, loading	The UI changes when Firebase reports a different user.
Inventory	products, loading	Firestore snapshots replace the visible item list.
Forms	name, category, price, errors	Controlled inputs display the latest typed values.
Dashboard	search, filters, sortBy	Changing a control immediately recalculates the list.
Feedback	isSubmitting, isDeleting	Buttons and messages show asynchronous progress.

Functional updates: ItemVault sometimes passes a function to a setter, for example `setProducts(currentProducts => ...)`. This is safer when the next value depends on the previous value because React supplies the freshest state.

2. useEffect - synchronization with systems outside React

What it means: `useEffect` runs after rendering. It is appropriate for subscriptions, browser APIs, timers, network synchronization, or other side effects.

Why ItemVault uses it: The authentication provider subscribes to Firebase Auth. The product provider subscribes to a Firestore query. Navbar synchronizes the theme with the document and `localStorage`. `EditRequest` updates the browser tab title.

```
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (firebaseUser) => {
    setUser(firebaseUser);
    setLoading(false);
  });

  return unsubscribe;
}, []);
```

Source: `src/context/AuthContext.jsx`, lines 31-38

Dependency behavior: The empty array means the auth subscription is created when `AuthProvider` mounts. The returned `unsubscribe` function runs during cleanup.

Important: The Firestore effect depends on `user`, so it reconnects when the signed-in user changes. Returning `unsubscribe` is especially important because the app runs inside `React StrictMode` during development, where effects may be mounted and cleaned up more than once to expose unsafe logic.

Effect dependency examples

- `[]` in `AuthContext`: set up one auth listener for the mounted provider.
- `[user]` in `ProductContext`: rebuild the query for the current user's UID.
- `[theme]` in `Navbar`: write the latest light/dark value to the DOM and `localStorage`.
- `[item]` in `EditRequest`: update `document.title` when the loaded item changes.

```
useEffect(() => {
  document.documentElement.setAttribute("data-theme", theme);
  window.localStorage.setItem("theme", theme);
}, [theme]);
```

Source: `src/components/Navbar.jsx`, lines 17-24

3. createContext, useContext, and custom hooks

createContext creates a shared channel. A Provider supplies a value, and useContext reads that value from descendants. ItemVault uses two channels:

- **AuthContext:** current user, loading state, login, Google login, signup, logout, and greeting data.
- **ProductContext:** products, loading state, CRUD actions, and item lookup.

```
const AuthContext = createContext(null);

export default function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used inside an AuthProvider");
  }
  return context;
}
```

Source: src/context/AuthContext.jsx and src/hooks/useAuth.jsx, lines 13 and 4-10

Why custom hooks? Components call useAuth() or useProducts() instead of importing context objects directly. This shortens consumer code and produces a clear error if a component is rendered outside its provider.

Provider placement

AuthProvider wraps the router, so authentication is available to Navbar and every route. ProductProvider is mounted only inside the protected product area, after ProtectedRoute confirms a user exists.

```
<AuthProvider>
  <BrowserRouter>...</BrowserRouter>
</AuthProvider>

<ProtectedRoute>
  <ProductProvider>
    <Outlet />
  </ProductProvider>
</ProtectedRoute>
```

Source: src/App.jsx and src/routes/ProtectedProductArea.jsx, lines 20-43 and 5-12

Why not pass props through every component? Authentication and product data are needed by many unrelated routes. Context avoids passing the same values through intermediate components that do not use them.

4. useMemo - reuse calculated values

What it means: useMemo stores the result of a calculation until one of its dependencies changes. It is useful for derived arrays or stable provider value objects.

Why ItemVault uses it: Products uses it to build category/location options and to filter and sort inventory. Both providers use it to avoid creating a new context value object when the relevant inputs have not changed.

```
const filteredItems = useMemo(() => {
  const matchingItems = products.filter((item) => {
    // search and filter checks
  });
  return [...matchingItems].sort(/* selected sort */);
}, [products, search, categoryFilter,
  statusFilter, locationFilter, sortBy]);
```

Source: src/pages/Products.jsx, lines 48-77

Dependency behavior: Any product or dashboard control that can change the result appears in the dependency array.

Important: useMemo is a performance optimization, not a replacement for state. The filtered list can be recalculated from existing state, so it should not be stored separately.

5. useCallback - stable function references

What it means: useCallback memoizes a function definition until its dependencies change.

Why ItemVault uses it: ProductProvider exposes functions through context. Stable function references help keep the memoized context value stable and reduce unnecessary updates in consumers.

```
const getProductById = useCallback(
  (id) => products.find((product) => product.id === id),
  [products]
);

const value = useMemo(
  () => ({ products, loading, getProductById, ...actions }),
  [products, loading, getProductById, ...actions]
);
```

Source: src/context/ProductContext.jsx, lines 160-168

Dependency behavior: getProductById changes when products changes. deleteProduct changes when products or user changes. updateProduct has no changing React dependencies, so it uses an empty array.

6. useRef - mutable values that do not render

What it means: `useRef` creates a stable object with a mutable `current` property. Updating it does not trigger a render.

Why ItemVault uses it: `ProductForm` uses a ref as an immediate duplicate-submission lock. `ProductProvider` uses a `Set` in a ref to remember optimistic deletions while `Firestore` snapshots catch up.

```
const isSubmittingRef = useRef(false);

if (isSubmittingRef.current ||
    Object.keys(validationErrors).length > 0) return;

isSubmittingRef.current = true;
setIsSubmitting(true);
```

Source: `src/components/ProductForm.jsx`, lines 35 and 55-64

Important: `isSubmitting` state controls the visible disabled/loading UI. The ref performs the instant logic guard. Using both is intentional because a state update is asynchronous, while `ref.current` changes immediately.

```
const pendingDeleteIds = useRef(new Set());

pendingDeleteIds.current.add(id);
setProducts((currentProducts) =>
  currentProducts.filter((product) => product.id !== id)
);
```

Source: `src/context/ProductContext.jsx`, lines 21 and 99-102

The pending deletion `Set` is internal bookkeeping. Rendering it would add no value, so state would cause unnecessary renders. `Products` state is still updated separately because the visible list must change.

7. React Router hooks

Hook	Where used	Purpose
<code>useNavigate</code>	Login, SignUp, AddProduct, EditRequest, ProductDetails	Moves to another route after login, save, update, or delete.
<code>useParams</code>	ProductDetails, EditRequest	Reads the item id from <code>/items/:id</code> .
<code>useLocation</code>	Login, ProductDetails, RequireAuth	Reads route state and remembers the page requested before login.

```
const { id } = useParams();
const location = useLocation();
const navigate = useNavigate();

const item = getProductById(id);
```

Source: `src/pages/ProductDetails.jsx`, lines 12-18

8. Related React patterns used by ItemVault

React.lazy

Loads each route component only when that route is needed.

Suspense

Displays a loading fallback while a lazy route module downloads.

StrictMode

Adds development checks and helps reveal unsafe effect behavior.

Controlled inputs

Input values come from state and update through onChange handlers.

Functional updates

Setter callbacks safely calculate next state from current state.

Optimistic UI

Deleted items disappear immediately and return if the request fails.

Source: src/App.jsx, src/main.jsx, src/components/ProductForm.jsx, src/context/ProductContext.jsx, lines 1-43, 6-9, 20-35, and 99-157

9. Hooks that are not currently used

Not every React hook improves every project. These hooks are absent because the current application does not have a clear need for them:

Hook	Why it is not necessary now
useReducer	State transitions are still understandable with individual state variables and provider actions.
useLayoutEffect	The app does not need synchronous DOM measurement before paint.
useId	Current labels and error IDs are static; reusable generated IDs are not required yet.
useTransition	Inventory filtering is small enough that urgent and non-urgent updates do not need separation.
useDeferredValue	Search remains responsive without delaying the value passed to expensive rendering.

Decision rule: Add a hook when it solves a concrete state, lifecycle, sharing, performance, or routing problem. Do not add memoization or advanced hooks only because they exist.

10. Practical rules followed in this project

- **Call hooks at the top level.** They are not placed inside conditions, loops, or event handlers.
- **Use state for rendered data.** Form values, loading indicators, errors, and product lists affect the UI.
- **Use refs for internal mutable bookkeeping.** Submission locks and pending deletion IDs do not need renders.
- **Clean up subscriptions.** Firebase Auth and Firestore listeners return unsubscribe functions from effects.
- **List effect and memo dependencies.** Values used by the calculation are represented in dependency arrays.
- **Use custom hooks to protect context access.** `useAuth` and `useProducts` fail clearly outside their providers.
- **Prefer derived values over duplicate state.** Filtered inventory is calculated from products and controls.
- **Keep provider functions stable where useful.** Product actions use `useCallback` and the value object uses `useMemo`.

A simple way to choose the right hook

Question	Use
Must changing this value update what the user sees?	<code>useState</code>
Must React synchronize with Firebase, the DOM, storage, or another external system?	<code>useEffect</code>
Do many descendants need the same value?	<code>createContext</code> + <code>useContext</code>
Is a derived calculation worth reusing until inputs change?	<code>useMemo</code>
Does a shared function need a stable identity?	<code>useCallback</code>
Must a mutable value survive renders without causing one?	<code>useRef</code>
Does the component need URL data or navigation?	React Router hooks

Summary: ItemVault's hook usage follows a clear separation. Providers own shared Firebase state and actions; components own temporary interface state; custom hooks expose shared context safely; memoization supports derived inventory data and stable context values; refs handle immediate mutable guards; and router hooks connect item screens to the URL.